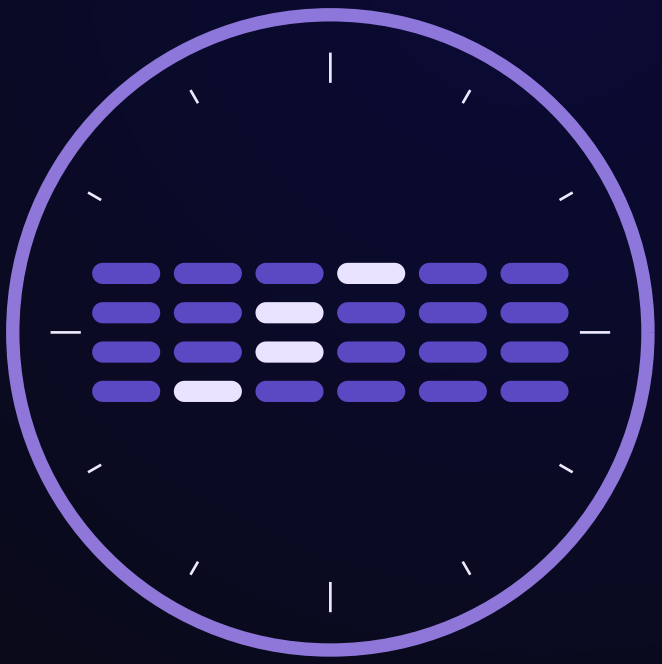
BookingApp



DOSSIER DEL PROYECTO · PORTFOLIO

Tus reservas, del caos al orden.

Plataforma SaaS de reservas multi-tenant para negocios de servicios: peluquerías, gimnasios, clínicas y espacios de coworking.

4	9	31	14
roles con políticas	controladores REST	tests automáticos	pantallas

ASP.NET Core 10 · EF Core · PostgreSQL · React 19 · TypeScript · SignalR · Hangfire · Stripe · three.js

Anass El Morabit

github.com/ANASSELMORABIT/BookingApp · Octubre 2026

01 · RESUMEN

Qué es BookingApp

Una plataforma web en la que cada negocio gestiona sus locales, su equipo, sus servicios y sus citas, y publica una página de reservas para que sus clientes reserven en segundos, sin crear cuenta.

El problema. Muchos negocios pequeños siguen gestionando las citas por teléfono, WhatsApp y papel: se pierden huecos, se solapan reservas y no hay datos para decidir.

La solución. Una agenda única y en tiempo real. El negocio define horario, servicios y personas; el sistema calcula los huecos libres, impide solapes, avisa al instante de cada reserva y recuerda las citas. Todo con datos aislados por negocio (multi-tenant) y permisos por rol.

Para el negocio
Panel con dashboard, agenda, calendario, gestión de negocios, equipo y servicios, estadísticas y exportación.

Para el cliente
Página pública /book/tu-negocio: elige servicio, persona, día y hora, y reserva sin registrarse.

Tiempo real
Notificaciones con SignalR al crear o cancelar reservas, sin recargar la página.

Automatización
Job de Hangfire cada hora que localiza las citas de las próximas 24 h para enviar recordatorios.

Mi papel
Proyecto personal de portfolio, diseñado y desarrollado de principio a fin: modelo de dominio, API, seguridad, base de datos, frontend, diseño de interfaz, tests y CI.

02 · FUNCIONALIDADES

Qué puede hacer

Gestión y operación

- **Multi-tenant:** cada negocio es un tenant identificado por un *slug*; el `tenant_id` viaja en el JWT y los servicios comprueban que cada negocio, servicio, empleado y reserva pertenece al tenant del usuario.
- **Roles y permisos:** SuperAdmin, Owner, Staff y Client, con autorización por políticas en la API y rutas protegidas por rol en el frontend.
- **Negocios, equipo y servicios:** alta y edición, horario de apertura, duración y precio de cada servicio, activar o desactivar personas y servicios.
- **Motor de disponibilidad:** genera los huecos del día según horario y duración, marca los ocupados por solapamiento y convierte entre UTC y la zona horaria del negocio.
- **Validación común de reservas** (autenticada y pública): mismo negocio, dentro del horario, no en el pasado y sin solapes.

Visualización y datos

- **Dashboard** con KPIs (hoy, semana, mes, ingresos, cancelaciones), rendimiento del equipo y servicios más populares.
- **Agenda** con tarjeta “Ahora” (cita en curso con su progreso o la próxima con cuenta atrás) y las reservas como línea de tiempo por día, con resumen y filtros.
- **Reservas**: el negocio ve todas las de sus clientes, también las de la página pública, separadas en próximas y anteriores, con búsqueda.
- **Calendario** de mes, semana y día (FullCalendar): cada reserva con el color de su estado, resumen de lo visible y detalle en un diálogo accesible.
- **Estadísticas** con un **horizonte 3D de reservas por día** (valores exactos al pasar el ratón y vista de tabla), gráficas animadas con paleta validada para daltonismo, y **exportación** a Excel (ClosedXML) y PDF (QuestPDF).

Clientes y difusión

- **Reserva pública** en cuatro pasos: servicio y persona → fecha y hora → datos → confirmación.
- **Página pública para compartir:** enlace del negocio con copiar, compartir y abrir, vista previa de lo que ve el cliente y lista de comprobación antes de difundirlo.
- **Página de inicio** con una escena 3D ligada al scroll en la que las citas pasan del desorden a una agenda ordenada.

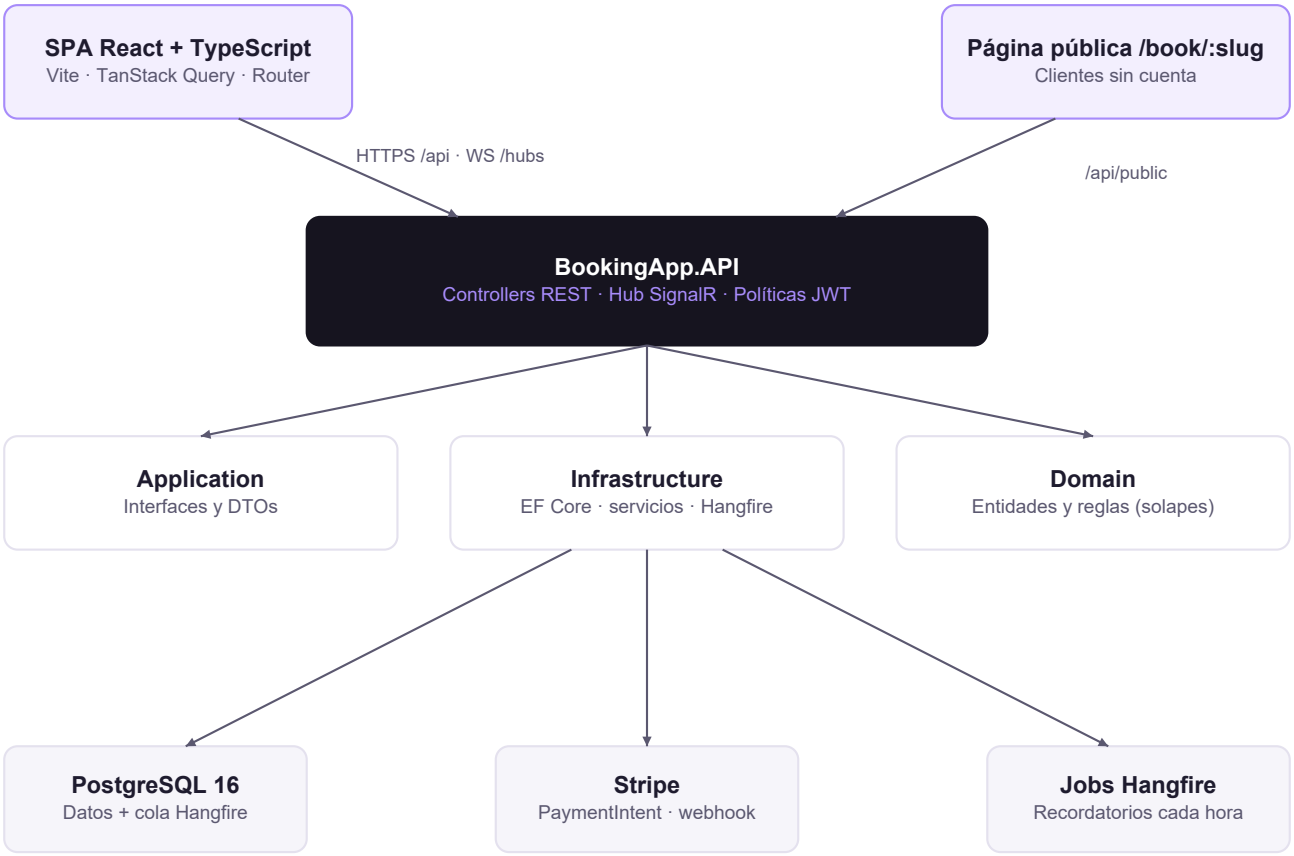
Experiencia de uso

- Menú lateral con indicador deslizante y contador de reservas de hoy; se pliega a solo iconos en escritorio y pasa a menú desplegable en móvil.
- Negocios con horario sobre una barra de 24 h y “abierto o cerrado ahora”; servicios con búsqueda, orden e interruptor animado; perfil con medidor de seguridad de la contraseña y barra de cambios sin guardar.
- Estados de reserva y pago en español y con los mismos colores en toda la app; confirmación antes de cancelar; avisos de éxito o error; esqueletos de carga; estados vacíos con acción; página 404.
- Accesibilidad: respeto a *reducir movimiento*, etiquetas en botones de icono, foco visible y contraste revisado en textos pequeños.

03 · ARQUITECTURA

Cómo está construido

Clean Architecture en el backend, con las dependencias apuntando hacia el dominio, y una SPA que consume la API.



Capa	Responsabilidad
Domain	Entidades (Tenant, Business, Service, Staff, Booking, AppUser) con estado encapsulado: setters privados, métodos de fábrica (Booking.Create) y reglas como Booking.OverlapsWith.
Application	Contratos de los servicios (interfaces) y DTOs de entrada y salida. No depende de EF Core ni de ASP.NET.
Infrastructure	DbContext, configuraciones y migraciones de EF Core, implementación de los servicios, Stripe, exportaciones y jobs de Hangfire.
API	Controllers REST, hub de SignalR, políticas de autorización y composición en Program.cs.
Frontend	React 19 + TypeScript: clientes de API por área, TanStack Query para el estado del servidor, componentes comunes y pantallas por área.

04 · TECNOLOGÍAS

Stack

Área	Tecnologías
Backend	ASP.NET Core 10 (Web API), C#, Entity Framework Core 10, Npgsql
Base de datos	PostgreSQL 16, migraciones con EF Core
Seguridad	JWT Bearer, BCrypt.Net, autorización por políticas, user-secrets
Tiempo real y jobs	SignalR, Hangfire (almacenamiento en PostgreSQL)
Integraciones	Stripe.net, ClosedXML (Excel), QuestPDF (PDF)
Documentación de la API	OpenAPI + Scalar
Frontend	React 19, TypeScript, Vite, React Router 7, TanStack Query, Axios
Interfaz	Tailwind CSS 4, shadcn/ui (Radix), Recharts, FullCalendar, Sonner, lucide
3D	three.js, React Three Fiber, drei (página de inicio y Estadísticas, solo en escritorio)
Calidad	xUnit, EF Core InMemory, ESLint, GitHub Actions
Despliegue	Dockerfile multi-stage para la API y docker-compose con PostgreSQL

05 · DECISIONES TÉCNICAS

Lo que hay detrás

Aislamiento por tenant
El tenant sale del JWT, nunca del cliente. Los servicios validan la pertenencia de cada recurso y el hub de SignalR asigna los grupos en el servidor a partir de los claims.

Fechas en UTC
Las reservas se guardan en UTC y se convierten a la zona del negocio (Europe/Madrid) al calcular y devolver huecos, evitando errores en los cambios de hora.

Reglas en el dominio
El solapamiento y los cambios de estado viven en la entidad Booking, y la validación es la misma para la reserva con cuenta y la pública.

Secretos fuera del repo
Cadena de conexión, clave JWT y claves de Stripe vienen de user-secrets o variables de entorno; la API no arranca si faltan.

3D sin penalizar la carga
La escena va en un archivo aparte que solo se descarga en escritorio, empieza a cargar en paralelo al primer pintado y se pausa fuera de pantalla. Versión estática para móvil y movimiento reducido.

Estado del servidor
TanStack Query gestiona caché, recargas y invalidaciones tras cada cambio, con lo que la interfaz siempre refleja los datos actuales.

Disponibilidad siempre fresca
Los huecos libres nunca se leen de caché y cada aviso en tiempo real recarga calendario, agenda y listas, para no ofrecer una hora que otro cliente acaba de reservar.

Datos sin trampas visuales
El 3D se usa en un solo elemento con los valores exactos a mano; las medidas de escala distinta van en gráficas separadas y la paleta está validada para daltonismo.

06 · PRODUCTO

Pantallas

Ruta	Pantalla	Acceso
/	Página de inicio con escena 3D	Público
/login · /register	Inicio de sesión y registro del negocio	Público
/book/:slug	Reserva pública en cuatro pasos	Público
/dashboard	KPIs, rendimiento del equipo y servicios más populares	Owner, Staff
/agenda	Cita actual o próxima y línea de tiempo por día	Owner, Staff
/calendar	Calendario de mes, semana y día	Owner, Staff
/bookings	Reservas del negocio y propias, y nueva reserva	Owner, Staff, Client
/business · /services	Negocios, equipo y servicios	Owner (servicios también Staff)
/stats	Horizonte 3D, gráficas y exportación a Excel y PDF	Owner
/share	Enlace público de reservas y vista previa	Owner
/profile	Perfil, contraseña y sesión	Todos

API principal

Método	Endpoint	Acceso	Uso
POST	/api/Auth/register · /login	Público	Registro del negocio (tenant + Owner) y login con JWT
GET/PUT	/api/Profile	JWT	Ver y editar perfil, cambiar contraseña
GET/POST/PUT	/api/Business…	JWT / Owner	Negocios, equipo y servicios
GET	/api/Availability	Público	Huecos disponibles
POST/GET/DELETE	/api/Bookings…	JWT / Owner, Staff	Crear, listar por cliente o negocio y cancelar
GET	/api/Admin/dashboard…	Owner, Staff	Estadísticas y series por periodo
GET	/api/Export/excel · /pdf	Owner	Exportaciones
GET/POST	/api/public/…	Público	Datos del negocio, disponibilidad y reserva sin cuenta
POST	/api/Payments/…	JWT · Stripe	PaymentIntent y webhook firmado
WS	/hubs/notifications	JWT	Notificaciones en tiempo real

07 · CALIDAD

Tests, CI y seguridad

31 tests con xUnit que se ejecutan en GitHub Actions en cada push, junto con el lint y el build del frontend:

- **Dominio:** solapamiento entre reservas y cambios de estado.
- **Reservas** (EF Core InMemory): conversión hora local ↔ UTC, solapamientos, reservas consecutivas, horario del negocio, fechas pasadas, reserva pública y aislamiento entre tenants.
- **Disponibilidad:** huecos de toda la jornada (incluido el último), huecos ocupados y empleados de otro negocio.
- **Registro y login:** slugs reservados o inválidos, imposibilidad de unirse a un tenant existente y mensajes de error que no revelan qué datos existen.
- **Flujo de la reserva pública:** la franja se bloquea solo para el empleado elegido (también para servicios de otra duración), repetir la hora se rechaza y el negocio ve la reserva.

Seguridad

- Contraseñas con BCrypt y tokens JWT con emisor, audiencia y caducidad validados.
- El registro siempre crea un tenant nuevo con rol Owner; los identificadores reservados (superadmin, admin…) no se pueden registrar.
- Webhook de Stripe con verificación de firma.
- Secretos fuera del repositorio (user-secrets en desarrollo, variables de entorno en Docker).

08 · ESTADO

Qué falta y siguientes pasos

En desarrollo
Emails: el servicio registra los envíos en el log; falta conectar un proveedor SMTP.
Pagos: el backend crea el PaymentIntent y procesa el webhook; falta el flujo de pago en el frontend.
Docker: API y PostgreSQL contenedorizados; el frontend se ejecuta con Vite.

- Proveedor de email real para confirmaciones y recordatorios.
- Pago con Stripe Elements en la reserva.
- Endpoints para que el negocio confirme, complete o cancele reservas de sus clientes, con sus botones en la Agenda.
- Filtro por negocio en la serie de reservas por periodo.
- Restricción de exclusión en PostgreSQL contra reservas solapadas en peticiones simultáneas, y filtros globales de EF Core por tenant.

Probarlo en local

Paso	Comando
1. Base de datos	`docker compose up -d postgres`
2. Migraciones	`dotnet ef database update --project src/BookingApp.Infrastructure --startup-project src/BookingApp.API`
3. Backend	`dotnet run --project src/BookingApp.API` → http://localhost:5239
4. Frontend	`cd src/Booking.Frontend && npm install && npm run dev` → http://localhost:5173

Los secretos se configuran con `dotnet user-secrets` (cadena de conexión, clave JWT y, opcionalmente, Stripe). El README del repositorio tiene la guía completa.